AWS CloudWatch EC2 Monitoring Lab

Complete Practical Guide: From Zero to 100%

Objective: Deploy a website on an Amazon EC2 instance, configure Amazon CloudWatch to collect server metrics and application logs, create a CPU alarm, and build a monitoring dashboard.

Operating system: Amazon Linux 2023
Web server: Nginx
Monitoring service: Amazon CloudWatch
Metrics collection: CloudWatch Agent
AWS region: US East (N. Virginia) — `us-east-1`

PART 1 — Understand the Lab

1. What is Amazon CloudWatch?

Amazon CloudWatch is an AWS monitoring service that helps you understand the health and performance of your applications and infrastructure.

Imagine you have a website running on an EC2 instance. Visitors are opening pages, the server is using memory, and requests are arriving continuously.

Without monitoring, you might not know when:

- CPU usage becomes too high.
- Memory usage increases.
- Disk space becomes full.
- The web server starts generating errors.
- The website stops responding.
- Suspicious or unexpected requests reach the server.

CloudWatch helps you collect information, display it visually, and create alarms when specific conditions occur.

2. What will this lab accomplish?

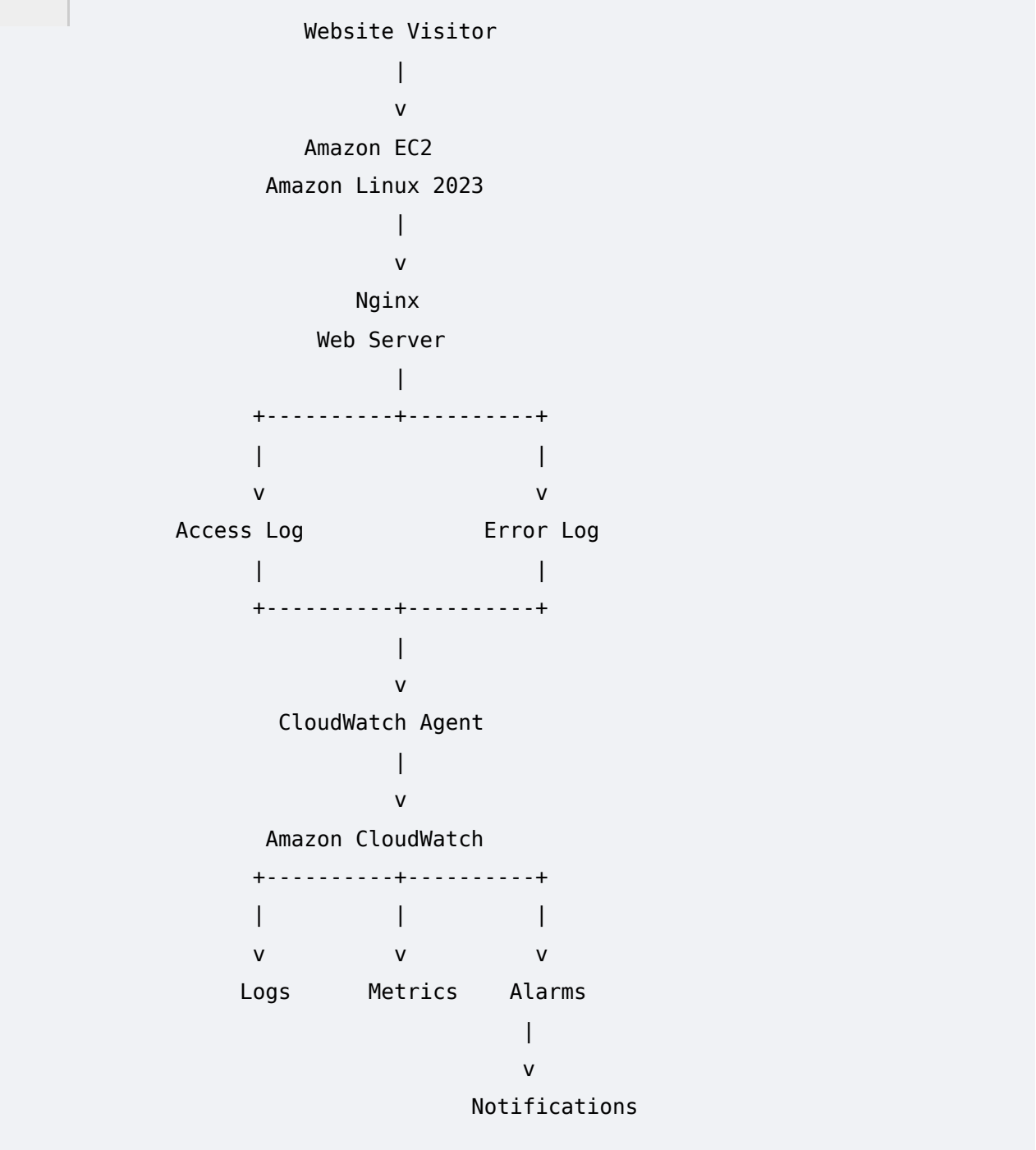
By the end of the lab, you will have:

1. An EC2 instance running Amazon Linux 2023.
2. An Nginx web server hosting a website.
3. An IAM role that allows the EC2 instance to send monitoring data to AWS.

4. The CloudWatch Agent installed and configured.
5. Nginx access and error logs delivered to CloudWatch Logs.
6. Memory and disk usage metrics published to CloudWatch.
7. CPU and network metrics collected from EC2.
8. A CPU utilization alarm.
9. A monitoring dashboard containing the most important metrics.
10. A working method to troubleshoot missing logs and metrics.

3. Understand the architecture

The monitoring process follows this path:



```
                    CloudWatch Dashboard
                    Displays key metrics
```

The CloudWatch Agent collects information from inside the operating system. CloudWatch also receives standard EC2 metrics directly from AWS without requiring the agent.

PART 2 — Create the EC2 Instance

4. Open the EC2 service

1. Sign in to the AWS Management Console.
2. Search for EC2.
3. Open the EC2 service.
4. Select the AWS region US East (N. Virginia).

The region matters because EC2 instances, CloudWatch metrics, log groups, alarms, and dashboards are region-specific.

5. Launch an EC2 instance

Navigate to:

EC2 → Instances → Launch instances

Configure the following settings.

Setting	Value
Name	`zelos-cloudwatch-lab`
Operating system	Amazon Linux 2023
Architecture	64-bit x86

Instance type	`t3.micro`, if available and suitable
Key pair	Create or select an existing key pair
Storage	8 GiB or another suitable lab size
Network	Default VPC is acceptable for this lab

Choose an instance type and storage configuration appropriate for your account and budget.

Configure the security group

Create a security group named:

`zelos-cloudwatch-lab-sg`

Add these inbound rules:

Type	Port	Source	Purpose

SSH	22	My IP	Remote administration
HTTP	80	`0.0.0.0/0`	Public website

Only expose HTTPS on port 443 if you configure HTTPS for the website.

Security note: Never leave SSH open to the entire internet unless there is a specific, justified requirement. Restrict it to your trusted IP address.

Leave outbound access enabled as needed for package installation and AWS service communication.

Click Launch instance.

6. Connect to the instance

Wait until the instance reaches the `Running` state and its status checks pass.

Go to:

EC2 → Instances → Select your instance → Connect

You can use the EC2 Instance Connect browser terminal if supported, or connect using SSH from your computer.

For SSH, the command follows this format:

```
ssh -i /path/to/your-key.pem ec2-user@YOUR_PUBLIC_IP
```

Replace the key path and public IP with your actual values.

If your private key has overly broad permissions on Linux, correct them:

```
chmod 400 /path/to/your-key.pem
```

Once connected, verify the operating system:

```
cat /etc/os-release
```

Check the current user:

```
whoami
```

Expected user:

```
ec2-user
```

Update the available package information and install system updates if appropriate for your lab:

```
sudo dnf update -y
```

PART 3 — Install and Configure Nginx

7. Install Nginx

Nginx is a web server that receives HTTP requests and returns website content to visitors.

Install it:

```
sudo dnf install -y nginx
```

Start the service:

```
sudo systemctl start nginx
```

Enable it to start automatically after a reboot:

```
sudo systemctl enable nginx
```

Check its status:

```
sudo systemctl status nginx --no-pager
```

The expected state is:

```
active (running)
```

Test Nginx locally:

```
curl -I http://localhost/
```

A successful response usually contains:

```
HTTP/1.1 200 OK
```

If the command fails, inspect the service:

```
sudo journalctl -u nginx --no-pager -n 50
```

8. Create a simple website

The default website directory is:

```
/usr/share/nginx/html/
```

Create a new HTML page:

```
sudo nano /usr/share/nginx/html/index.html
```

Enter the following HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>CloudWatch Monitoring Lab</title>

    <style>
        * {
            box-sizing: border-box;
        }

        body {
            margin: 0;
            min-height: 100vh;
            display: grid;
            place-items: center;
            padding: 24px;
            font-family: Arial, sans-serif;
            background: #0b1118;
            color: #edf4f7;
        }

        .card {
            width: min(620px, 100%);
            padding: 36px;
            border: 1px solid #29423a;
            border-radius: 18px;
```

```
            background: #121d24;
        }

        h1 {
            color: #64f0a5;
        }

        .status {
            display: inline-block;
            padding: 8px 12px;
            border-radius: 20px;
            background: #153b2b;
            color: #64f0a5;
        }

        p {
            line-height: 1.7;
            color: #c1d0d8;
        }

        a {
            color: #64f0a5;
        }
    </style>
</head>

<body>
    <main class="card">
        <span class="status">Website Online</span>

        <h1>CloudWatch Monitoring Lab</h1>

        <p>
            This website is hosted on an Amazon EC2 instance
            running Amazon Linux 2023 and Nginx.
        </p>

        <p>
            Amazon CloudWatch will be used to monitor server
            metrics, collect web server logs, and generate alarms.
        </p>
```

```
            <p>
                <a href="/cloudwatch-test-page">
                    Open the monitoring test page
                </a>
            </p>
        </main>
    </body>
    </html>
```

Save in nano with `Ctrl+O`, press `Enter`, then exit with `Ctrl+X`.

Create a separate test page:

```
sudo nano /usr/share/nginx/html/cloudwatch-test-page
```

Enter:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>CloudWatch Test Page</title>
</head>
<body>
    <h1>CloudWatch Test Page</h1>
    <p>This page generates a test request for monitoring.</p>
</body>
</html>
```

Save and exit.

The test page is deliberately named without an `.html` extension. Nginx will serve this file at `/cloudwatch-test-page`.

Check the website:

```
curl -I http://localhost/
```

```
curl -I http://localhost/cloudwatch-test-page
```

Both should return a successful HTTP response, normally `200 OK`.

Now open this address in your browser:

```
http://YOUR_PUBLIC_IP
```

Replace `YOUR_PUBLIC_IP` with the instance's public IPv4 address.

If the website works locally but not in your browser, check the security group's inbound HTTP rule and verify that the instance has a public IP and suitable network routing.

PART 4 — Understand the Logs

9. What is a log file?

A log is a record of events that occurred in an application or operating system.

Nginx normally writes two important logs.

Access log

Path:

```
/var/log/nginx/access.log
```

The access log records incoming HTTP requests.

It commonly includes:

- Client IP address.
- Date and time.
- HTTP method, such as `GET`.
- Requested URL.
- HTTP status code.
- Response size.
- Browser or client information.

For example, a request might produce an entry similar to:

```
192.0.2.10 - - [28/Oct/2026:12:00:00 +0000] "GET / HTTP/1.1" 200 1250 "-
" "Mozilla/5.0"
```

The example IP address is reserved for documentation and is not a real visitor.

Error log

Path:

```
/var/log/nginx/error.log
```

The error log records Nginx problems and diagnostic information.

Examples include:

- Missing files.
- Permission errors.
- Failed upstream connections.
- Configuration problems.
- Other server-side issues.

View the logs:

```
sudo tail -n 20 /var/log/nginx/access.log
```

```
sudo tail -n 20 /var/log/nginx/error.log
```

Follow the access log as new requests arrive:

```
sudo tail -f /var/log/nginx/access.log
```

Press `Ctrl+C` to stop following the file.

10. Generate sample traffic

Run:

```
for i in {1..10}; do
    curl -s -o /dev/null http://localhost/
    sleep 1
done
```

Generate requests to the test page:

```
for i in {1..10}; do
    curl -s -o /dev/null http://localhost/cloudwatch-test-page
    sleep 1
done
```

Check the new entries:

```
sudo tail -n 30 /var/log/nginx/access.log
```

You should see requests to `/` and `/cloudwatch-test-page`.

This traffic will be useful later when you test CloudWatch Logs.

PART 5 — Create an IAM Role for CloudWatch

11. Why is an IAM role necessary?

An EC2 instance needs permission to send its monitoring information to AWS.

IAM stands for Identity and Access Management. It controls which users, applications, and AWS resources are allowed to perform specific actions.

An IAM role provides temporary credentials to an authorized AWS service or workload.

For this lab, the EC2 instance needs a role that permits the CloudWatch Agent to publish metrics and send log events.

Do not store permanent AWS access keys in your agent configuration.

12. Create the role

Open:

AWS Console → IAM → Roles → Create role

Configure:

- Trusted entity type: AWS service.
- Use case: EC2.

Continue to the permissions page.

Search for and attach the AWS managed policy:

```
CloudWatchAgentServerPolicy
```

Set the role name to:

```
EC2CloudWatchAgentRole
```

Create the role.

This policy supplies the permissions commonly required by the CloudWatch Agent. For production environments, consider a custom least-privilege policy tailored to the exact resources and actions required.

13. Attach the role to the EC2 instance

Navigate to:

EC2 → Instances → Select your instance

Choose the instance actions or security menu and select Modify IAM role.

Choose:

```
EC2CloudWatchAgentRole
```

Save the change.

Verify that the role appears in the instance's IAM role field.

14. Verify that EC2 can see the role

Run the following commands in the EC2 terminal.

Request an IMDSv2 token:

```
TOKEN=$(curl -sS -X PUT \
  "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

Ask the EC2 metadata service for the attached role name:

```
curl -sS \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/"
```

Expected output:

```
EC2CloudWatchAgentRole
```

If the response is empty or an error, verify the role attachment and metadata configuration before continuing.

Never share the output of the credentials endpoint, which contains temporary credentials rather than just the role name.

PART 6 — Install the CloudWatch Agent

15. What is the CloudWatch Agent?

The CloudWatch Agent is software installed inside an EC2 instance.

It can collect information that standard EC2 monitoring does not automatically provide, including:

- Operating system memory usage.
- Filesystem disk usage.
- Selected application log files.
- Additional operating system metrics, depending on configuration.

It sends the collected information to CloudWatch.

16. Install the agent

On Amazon Linux 2023, run:

```
sudo dnf install -y amazon-cloudwatch-agent
```

Check the installed package:

```
rpm -q amazon-cloudwatch-agent
```

The package should be listed as installed.

Find the control utility:

```
ls -l /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl
```

The main paths used in this lab are:

Purpose	Path
Agent configuration	`/opt/aws/amazon-cloudwatch-agent/bin/config.json`
Agent log	`/opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-`

	`agent.log`
Control utility	`/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl`
Nginx access log	`/var/log/nginx/access.log`
Nginx error log	`/var/log/nginx/error.log`

PART 7 — Configure the CloudWatch Agent

17. Understand the configuration

The configuration file tells the agent:

1. How frequently to collect metrics.
2. Which metrics to collect.
3. Which CloudWatch namespace should contain custom metrics.
4. Which log files should be sent to CloudWatch Logs.

5. Which log groups and log streams should receive the data.

A namespace is a container used to organize metrics. We will use:

```
Zelos/EC2
```

A log group is a collection of related log streams. We will use separate groups for Nginx access and error logs.

A log stream is a sequence of log events, often associated with a specific source. In this lab, the stream name will use the EC2 instance ID.

18. Create the configuration file

Run:

```
sudo nano /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Enter the following complete configuration:

```
{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": "root"
  },
  "metrics": {
    "namespace": "Zelos/EC2",
    "append_dimensions": {
      "InstanceId": "${aws:InstanceId}"
    },
    "metrics_collected": {
      "mem": {
        "measurement": [
          "mem_used_percent"
        ],
        "metrics_collection_interval": 60
      },
      "disk": {
        "measurement": [
          "used_percent"
        ],
        "resources": [
          "/"
        ],
```

```
            "metrics_collection_interval": 60
          }
        }
      },
      "logs": {
        "logs_collected": {
          "files": {
            "collect_list": [
              {
                "file_path": "/var/log/nginx/access.log",
                "log_group_name": "/zelos/nginx/access",
                "log_stream_name": "{instance_id}"
              },
              {
                "file_path": "/var/log/nginx/error.log",
                "log_group_name": "/zelos/nginx/error",
                "log_stream_name": "{instance_id}"
              }
            ]
          }
        }
      }
    }
```

Save the file using `Ctrl+O`, press `Enter`, and exit with `Ctrl+X`.

19. Validate the JSON

A JSON configuration must be syntactically valid. A missing comma or quotation mark can prevent the agent from loading it.

Run:

```
python3 -m json.tool /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

If the file is valid, the command prints formatted JSON.

If it reports an error, return to the configuration file and correct the line identified by Python.

20. Understand the metrics in this configuration

Memory metric

```
mem_used_percent
```

This measures the percentage of memory reported as used by the operating system.

High memory usage can indicate that an application is consuming excessive resources. It is important to understand the instance's normal behavior before deciding what threshold should trigger an alert.

Disk metric

```
used_percent
```

This measures the percentage of the selected filesystem in use.

The configuration specifies:

```
/
```

This means the root filesystem.

If disk usage approaches 100%, the instance may be unable to write logs, store files, or perform normal operations.

Collection interval

```
60 seconds
```

The agent collects these configured metrics every 60 seconds.

The CloudWatch metric may take additional time to appear after the agent starts.

PART 8 — Start the CloudWatch Agent

21. Load the configuration and start the agent

Run this command:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config \
  -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
  -s
```

The options mean:

- `-a fetch-config` : load the supplied configuration.
- `-m ec2` : run the agent in an EC2 environment.

- `-c file:...`: specify the local configuration file.
- `-s`: start the agent after loading the configuration.

22. Check the agent status

Run:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
```

A successful status response should contain:

```
{
  "status": "running",
  "configstatus": "configured"
}
```

Check the service:

```
sudo systemctl status amazon-cloudwatch-agent --no-pager
```

If you make configuration changes later, load the updated file again using the `fetch-config` command.

23. Inspect the agent log

Run:

```
sudo tail -n 60 /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log
```

Look for messages indicating that the agent started and is processing data.

For log forwarding, you may see messages indicating that the agent is piping files such as:

```
/var/log/nginx/access.log
/var/log/nginx/error.log
```

to the configured CloudWatch log groups.

Check for errors involving:

- AWS credentials.
- IAM permissions.
- Invalid JSON.

- Missing log files.
- Network connectivity.
- Failed requests to AWS endpoints.

A running agent does not guarantee successful delivery. The next steps verify the actual data in CloudWatch.

PART 9 — Verify CloudWatch Logs

24. Generate new log entries

Run:

```
for i in {1..10}; do
    curl -s -o /dev/null http://localhost/
    sleep 1
done
```

Then:

```
for i in {1..10}; do
    curl -s -o /dev/null http://localhost/cloudwatch-test-page
    sleep 1
done
```

Check the local access log:

```
sudo tail -n 30 /var/log/nginx/access.log
```

This confirms that Nginx is producing entries for the agent to collect.

25. Open CloudWatch Log groups

In the AWS Console:

1. Search for CloudWatch.
2. Open CloudWatch.
3. Confirm that the region is `us-east-1`.
4. Open Logs → Log groups.
5. Find the following groups:

```
/zelos/nginx/access
/zelos/nginx/error
```

If the log groups do not appear immediately, wait a few minutes and refresh.

If they remain missing, inspect the agent log and check the role, policy, configuration, and log file paths.

26. Inspect a log stream

Open:

```
/zelos/nginx/access
```

Select the stream named with the EC2 instance ID.

You should find log events containing HTTP requests.

For example, requests may include:

```
GET /
GET /cloudwatch-test-page
```

The actual entries will also contain timestamps, status codes, and other Nginx access-log fields.

27. Search logs with CloudWatch Logs Insights

CloudWatch Logs Insights lets you search, filter, and analyze log events.

Navigate to:

CloudWatch → Logs → Logs Insights

Select the log group:

```
/zelos/nginx/access
```

Set the time range to Last 15 minutes and run:

```
fields @timestamp, @message
| filter @message like /cloudwatch-test-page/
| sort @timestamp desc
| limit 20
```

This query works as follows:

- `fields` selects the fields to display.
- `@timestamp` is the event timestamp.
- `@message` contains the raw log entry.
- `filter` keeps entries containing the requested path.
- `sort` orders events with the newest first.

- `limit` restricts the result count.

If the query returns no rows, try a broader query:

```
fields @timestamp, @message
| sort @timestamp desc
| limit 30
```

Check the selected log group, region, and time range before concluding that the data is missing.

PART 10 — Verify CloudWatch Metrics

28. What is a metric?

A metric is a numerical measurement collected over time.

Examples include:

- CPU utilization.
- Memory utilization.
- Disk utilization.
- Network traffic.
- Failed EC2 status checks.

CloudWatch stores metric datapoints with timestamps and values, allowing you to visualize changes over time.

29. View standard EC2 metrics

Open:

CloudWatch → Metrics → All metrics → EC2

Select the per-instance metric category and find your EC2 instance.

Look for the following metrics:

Metric	Meaning
`CPUUtilization`	Percentage of allocated EC2 CPU capacity

	being used
`NetworkIn`	Incoming network bytes
`NetworkOut`	Outgoing network bytes
`StatusCheckFailed`	Indicates a failed instance or system status check

These standard metrics are available from EC2 without installing the CloudWatch Agent.

The exact set of standard metrics and their publication intervals depends on the AWS monitoring configuration.

30. View custom memory and disk metrics

Open:

CloudWatch → Metrics → All metrics → Custom namespaces

Select:

```
Zelos/EC2
```

Find the metrics:

```
mem_used_percent
used_percent
```

Choose the dimension set containing your EC2 instance ID.

Set the time range to Last 15 minutes or another recent interval.

If the metrics are not visible, allow several minutes after starting the agent, then check:

- The agent status.
- The JSON configuration.
- The agent log.
- The selected AWS region.
- The custom namespace.

31. Understand why these metrics matter

CPU utilization

High CPU usage can cause slower responses and reduced application performance.

Memory utilization

High memory usage can increase the risk of applications being terminated or the operating system running out of available memory.

Disk utilization

High disk usage can prevent logs and application files from being written.

Network traffic

Unexpected changes in incoming or outgoing traffic can indicate workload changes, connectivity issues, or unusual activity. Traffic alone does not prove that an attack is occurring.

Status checks

EC2 status checks help identify infrastructure or instance-level problems that can affect availability.

PART 11 — Create a CPU Utilization Alarm

32. What is a CloudWatch alarm?

A CloudWatch alarm monitors a metric and changes state when the configured conditions are met.

Common states include:

- `OK`: the metric is within the configured condition.
- `ALARM`: the condition has been met.
- `INSUFFICIENT_DATA`: there is not enough data to determine the state.

An alarm does not automatically repair a server. It can notify an administrator or trigger an appropriately configured action.

33. Create the alarm

Navigate to:

CloudWatch → Alarms → All alarms → Create alarm

Select Select metric.

Choose:

EC2 → Per-Instance Metrics

Find and select:

```
CPUUtilization
```

Choose the metric belonging to your EC2 instance.

Configure the following settings:

Setting	Value
Statistic	Average
Period	5 minutes
Threshold type	Static
Condition	Greater than
Threshold	70
Evaluation periods	2
Datapoints to alarm	2

This configuration is intended to alarm when CPU utilization exceeds 70% for two consecutive five-minute evaluation periods.

Review the graph and the selected metric before continuing.

34. Configure notifications

You can connect the alarm to an Amazon SNS topic to receive notifications.

SNS stands for Simple Notification Service.

If you want email notifications:

1. Create or select an SNS topic.
2. Add an email subscription.
3. Confirm the subscription using the email sent by AWS.
4. Select the topic as the alarm's notification action.

Without a confirmed email subscription, email alerts may not reach you.

If you do not want notifications in this lab, configure the alarm without an SNS notification action.

35. Name and create the alarm

Use this alarm name:

```
zelos-ec2-high-cpu
```

Review the settings and create the alarm.

Open the alarm details to verify that the correct EC2 metric and threshold are selected.

A low-traffic website may never exceed 70% CPU. The alarm can remain in the `OK` state, which is normal.

Do not run an aggressive stress test on a production server just to trigger the alarm.

PART 12 — Create a CloudWatch Dashboard

36. What is a dashboard?

A CloudWatch dashboard is a customizable page containing charts and other monitoring widgets.

Instead of opening each metric individually, you can view the key measurements for your EC2 instance in one place.

37. Create the dashboard

Navigate to:

CloudWatch → Dashboards → Create dashboard

Enter the name:

```
zelos-ec2-monitoring
```

Create the dashboard and begin adding widgets.

38. Add the CPU chart

Add a line chart widget.

Select:

EC2 → Per-Instance Metrics → CPUUtilization

Choose your EC2 instance.

Save the widget.

This chart will show how CPU usage changes over time.

39. Add network charts

Add widgets for:

```
NetworkIn
NetworkOut
```

Select the same EC2 instance.

These charts help you understand incoming and outgoing traffic.

Remember that these metrics are measured in bytes, not necessarily requests per second.

40. Add the memory chart

Add a line chart.

Navigate to:

Custom namespaces → Zelos/EC2

Select:

```
mem_used_percent
```

Choose the correct instance dimension.

Save the widget.

41. Add the disk chart

Add another line chart using:

```
used_percent
```

Choose the metric associated with your instance and root filesystem.

Save the widget.

42. Add the status-check metric

Add a chart for:

```
StatusCheckFailed
```

Choose the correct EC2 instance.

This helps identify failed EC2 status checks.

43. Arrange and save the dashboard

A useful dashboard layout is:

- First row: CPU utilization.
- Second row: Memory and disk usage.
- Third row: Network traffic and status checks.

Set the dashboard time range to Last 1 hour.

Save the dashboard and verify that the charts display data.

The dashboard is complete when all intended widgets are saved and the selected metrics correspond to the correct EC2 instance.

PART 13 — Troubleshooting

44. CloudWatch Agent is not running

Check the service:

```
sudo systemctl status amazon-cloudwatch-agent --no-pager
```

Check its recent logs:

```
sudo tail -n 80 /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log
```

Try restarting it:

```
sudo systemctl restart amazon-cloudwatch-agent
```

Check the status again:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a
status
```

If the agent fails to start, inspect the configuration and the error messages before trying further changes.

45. The log groups do not appear

Check the local log files:

```
sudo ls -l /var/log/nginx/access.log
sudo ls -l /var/log/nginx/error.log
```

Generate a fresh request:

```
curl -s -o /dev/null http://localhost/
```

Inspect the agent log:

```
sudo tail -n 80 /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-
agent.log
```

Confirm that:

- The correct IAM role is attached.
- The role has suitable CloudWatch permissions.
- The configuration uses the correct file paths.
- The agent is running.
- You are looking in the correct AWS region.

46. The agent reports missing AWS credentials

Verify that the role name is accessible through IMDSv2:

```
TOKEN=$(curl -sS -X PUT \
  "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

curl -sS \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/"
```

If the role name is missing, check the instance's IAM role attachment and metadata settings.

Do not paste temporary credential values into documents, chat messages, or screenshots.

47. Memory and disk metrics are missing

Validate the JSON:

```
python3 -m json.tool /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Reload the configuration:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config \
  -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
  -s
```

Check the status:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a
status
```

Wait a few minutes, then revisit the `Zelos/EC2` namespace in CloudWatch Metrics.

48. The website is inaccessible

Check Nginx:

```
sudo systemctl status nginx --no-pager
```

Test locally:

```
curl -I http://localhost/
```

Check the instance's security group to confirm that inbound HTTP traffic is allowed on port 80.

Also confirm that the instance has a public IP and an appropriate route to an internet gateway if the website is intended to be publicly accessible.

49. The CPU alarm never enters ALARM

A quiet instance may not generate enough CPU load to cross the threshold.

Verify:

- The correct instance was selected.

- The metric is `CPUUtilization`.
- The threshold is greater than 70%.
- The evaluation period is configured correctly.
- Recent datapoints are available.

An alarm remaining in `OK` is not a failure if CPU utilization has stayed below the threshold.

PART 14 — Security and Cost Management

50. Secure the EC2 instance

Before finishing the lab:

- Restrict SSH access to your trusted IP.
- Keep only the necessary inbound ports open.
- Use IAM roles instead of permanent AWS access keys.
- Apply system updates as appropriate.
- Review Nginx error logs for application issues.
- Avoid storing passwords, tokens, or secrets in log files.
- Review the permissions attached to the EC2 role.

51. Understand CloudWatch costs

CloudWatch may charge for features such as:

- Log ingestion.
- Log storage.
- Custom metrics.
- Alarms.
- Dashboards.
- Notifications and related services.

Actual charges depend on current AWS pricing, usage, region, and account eligibility for any free usage allowances.

Check current pricing before leaving resources running.

52. Configure log retention

Log groups can retain events indefinitely unless a retention policy is configured.

To reduce unnecessary storage, open:

CloudWatch → Logs → Log groups

Select each group:

```
/zelos/nginx/access
/zelos/nginx/error
```

Choose the retention settings and select a suitable period, such as 7 or 14 days for a temporary lab.

Use a longer retention period if required for your operational or compliance needs.

PART 15 — Final Verification

53. Verify every component

Use this checklist to confirm that the practical lab is complete.

EC2 and website

- EC2 instance is running.
- Nginx is active.
- The homepage returns a successful HTTP response.
- The test page returns a successful HTTP response.
- Access and error log files exist.

IAM and CloudWatch Agent

- The EC2 instance has an IAM role attached.
- The role has appropriate CloudWatch permissions.
- The CloudWatch Agent is installed.
- The configuration file is valid JSON.
- The agent status shows `running`.
- The agent status shows `configured`.

CloudWatch Logs

- `/zelos/nginx/access` exists.
- `/zelos/nginx/error` exists.
- Log streams contain actual Nginx events.
- Logs Insights returns recent request entries.

CloudWatch Metrics

- `CPUUtilization` is visible.
- `NetworkIn` and `NetworkOut` are visible.
- `StatusCheckFailed` is available.

- `mem_used_percent` appears under `Zelos/EC2`.
- `used_percent` appears under `Zelos/EC2`.

Alarms and dashboards

- `zelos-ec2-high-cpu` exists.
- The alarm monitors the correct instance.
- The CPU threshold and evaluation periods are correct.
- SNS notifications are configured and confirmed if required.
- `zelos-ec2-monitoring` exists.
- The dashboard contains CPU, memory, disk, network, and status-check widgets.
- The widgets display the correct instance's data.

Security and cleanup

- SSH access is restricted to a trusted IP.
- Log retention is configured appropriately.
- CloudWatch costs have been considered.
- Unneeded lab resources are stopped or removed.

Final Outcome

When all verification items pass, you will have built an EC2 monitoring setup that collects Nginx application logs, publishes operating system memory and disk metrics, displays standard EC2 metrics, evaluates a CPU alarm, and presents important measurements in a CloudWatch dashboard.

These are foundational monitoring skills used in cloud administration, operations, and cloud security engineering.